“RISC e CISC: Comparação de suas técnicas”

Alane Evellyn Dantas Silva (20260009364)
Kalebe Carvalho Domingos (20260009346)
Maria Laura Sarmento Veríssimo (20260009023)

1. INTRODUÇÃO

O artigo apresenta uma comparação entre as arquiteturas de processadores CISC (*Complex Instruction Set Computer*) e RISC (*Reduced Instruction Set Computer*), onde a arquitetura CISC possui um conjunto maior e mais complexo de instruções, enquanto a RISC utiliza um conjunto menor e mais simples, buscando executar as instruções de maneira mais rápida.

A arquitetura de um processador define a forma como ele executa as instruções utilizadas por um computador. O artigo explica que, inicialmente, com a arquitetura CISC, havia uma tendência de aumentar cada vez mais a complexidade dos conjuntos de instruções dos processadores. Entretanto, alguns fabricantes passaram a seguir uma abordagem diferente, dando origem à arquitetura RISC. Essa mudança buscava melhorar o desempenho por meio de instruções mais simples e de uma execução mais eficiente, onde os processadores RISC se tornaram uma opção mais simples e barata capaz de trabalhar com *clocks* mais altos.

Dessa forma, o objetivo do artigo é comparar as técnicas utilizadas pelas arquiteturas CISC e RISC, mostrando suas características, seu funcionamento e suas diferenças na execução das operações.

2. ENTENDENDO CISC e RISC

Nesta seção o artigo explica que as primeiras máquinas que seguiram a arquitetura CISC surgiram a partir da microprogramação, técnica mais usual na época de 1975 devido às tecnologias de memória existentes. Essas arquiteturas baseadas na microprogramação eram conhecidas por interagirem muito com a memória, o que significa dizer que precisavam de múltiplos ciclos de clock para executar as instruções completamente.

O artigo explica que isso acontece porque as instruções de máquina são interpretadas por um microprograma localizado em uma memória do circuito integrado do processador. Entretanto, um ponto importante a se destacar é que os acessos à memória são frequentes durante a execução, pois a arquitetura CISC possui uma pequena quantidade de registradores em relação às arquiteturas mais modernas.

Em anos posteriores, o documento explica que em uma onda na qual os processadores tendiam a ser mais complexos, alguns fabricantes optaram por buscar alta performance, e foi nesse cenário que surgiu a arquitetura RISC com a execução das instruções a cada ciclo de clock, formato fixo e implementação por pipeline.

O texto define pipeline como uma técnica de implementação a qual várias instruções são sobrepostas na execução. No geral, o objetivo é conseguir realizar mais de uma etapa ao mesmo tempo para reduzir o tempo total de execução.

3. COMPARAÇÃO DE OPERAÇÕES E CONCLUSÃO

O artigo apresenta a comparação da arquitetura CISC com a arquitetura RISC ao aplicar, em ambas as máquinas, uma multiplicação. Sob esse viés, a CISC analisada

executa a função de multiplicação (“MULT”) em uma linha, porque usa instruções robustas inscritas no seu hardware, de modo a ser menor, funcionando da seguinte maneira:

MULT 2: 2, 5: 5

I. Carrega dois valores em registradores separados;
II. Passa os valores para a Unidade de Execução;
III. Multiplica os valores;
IV. Passa a resposta final para a Memória Principal, em que serão armazenados.

Nesse viés, em razão de sua forma diminuta, a operação, em CISC, é fácil de ser traduzida para um compilador, pois não requer uso extenso de memória do computador para o armazenamento de instruções. Além disso, seus comandos acessam o banco de memória independentemente, sem a especificação clara do programador.

Em contraposição, a arquitetura RISC se constrói em operações simples, as quais usam somente um *clock*. Com isso, ela transforma o único comando em CISC, “MULT”, em quatro instruções, três delas distintas:

```
load a, 2:2
load b, 5: 5
mult a, b
store 2: 2, a
```

I. “load”: move as variáveis da memória para um registrador;
II. “mult”: encontra o produto dos operandos representados nas variáveis;
III. “store”: guarda esse resultado na memória.

Embora use mais linhas de código que o comando em CISC, repassando mais trabalho para o compilador, essa linha de operações leva aproximadamente o mesmo tempo, ao ocorrer em um *clock*. Ademais, diferente da arquitetura CISC, a RISC guarda as variáveis usadas nos registradores até que eles sejam necessitados para outros valores, em vez de descartá-las logo após terminar as instruções, de maneira que auxilia na reutilização dessas variáveis.

Por fim, a RISC tem uma abordagem dividida e simples, enquanto a CISC é complexa e objetiva em seus códigos. Dessarte, o artigo conclui que, mesmo ao apresentarem especificidades distintas, as arquiteturas exemplificadas trabalham melhor em conjunto, com suas melhores partes em harmonia.

“Comparação entre as arquitecturas de processadores RISC e CISC”

1. INTRODUÇÃO

O artigo descreve a relação histórica e a técnica entre as arquiteturas CISC (*Complex Instruction Set Computer)* e RISC (*Reduced Instruction Set Computer)*, suas evoluções através do tempo e uma visão atual sobre arquiteturas de computador na visão CISC / RISC. Ele narra a criação de cada arquitetura, seus objetivos, seus funcionamentos e uma comparação entre as funcionalidades de ambas.

Para se estabelecer conceitos básicos, primeiro, o artigo explica três vertentes que definiram as ideias em torno das arquiteturas citadas:

I. Memória e armazenamento: na época, um dos maiores desafios nessa vertente era a relação custo-rapidez de memórias, que, quando rápidas, para os padrões do tempo, custavam demais. Além disso, ainda havia o problema do armazenamento secundário, o qual também costumava ser caro, mas lento. Assim, visar ao uso diminuto de armazenamento / de memória em programas era uma solução;
II. Compiladores: o mínimo esperado de um compilador era a tradução correta, mas, para se obter uma tradução ótima de programas escritos em linguagens de alto nível (*C*, *pascal*) para *assembly*, era necessária a programação direta nessa linguagem, para ser convertida em linguagem de máquina. Assim, os compiladores disponíveis eram pobres;
III. VLSI (*Very Large Scale Integration)*: as limitações do tempo definiram uma arquitetura com vários *chips*, em que se dividiram os transístores. Esse fato aumentava o tempo de espera, em razão à movimentação de dados entre *chips* necessária para a realização de comandos, de modo que havia um problema de desempenho causado por esse óbice.

2. CISC

Nesse prisma, havia tendências contrárias entre *software* e *hardware*: o *software* ficava paulatinamente mais caro, enquanto o preço da montagem de *hardware* descia. Esse cenário trouxe uma troca entre esses dois componentes, na qual elementos que, antes, seriam deixados para o *software*, iriam começar a ser programados diretamente em *hardware*, com funções mais robustas, mas já embutidas na máquina. Essa filosofia montou a arquitetura CISC, que, como em seu nome, usa instruções complexas, ao incluir múltiplos comandos em *assembly* em somente uma linha de código, no exemplo do comando “MULT”.

Sob esse viés, alguns dos objetivos de fazer uma arquitetura focada em complexidade eram trazer simplificação para os programadores, em um nível que mantivesse um preço viável para máquinas. A CISC diminui a dificuldade de revisão de código e ajuda na compactação e na facilidade em torno da escrita de programas, de maneira que reduz o preço global de sistemas. Além disso, com uma diminuição das instruções colocadas em sistemas, haveria um aumento no desempenho geral, ao analisar a comparação entre número de instruções e tempo gasto, embora seus comandos ainda demorassem mais que um ciclo de *clock*, comumente.

Ademais, outra característica marcante da arquitetura CISC é seu modo de endereçamento complexo. Em vez de demandar vários comandos, para fazer operações, a CISC usa um comando que funde vários outros comandos em *assembly*.

```
MOV [A, 2:3]
MOV [B, 5:2]
MUL [A, B]
MOV [2:3, A]
```

I. “MOV”: pega variáveis das posições de memória [2:3] e [5:2] para os registradores A e B;
II. “MUL”: multiplica os conteúdos de A e de B.

Por fim, a arquitetura também usa micro-códigos para diminuir o uso de espaço. A fim de explicar a micro-programação, antes, o artigo disserta sobre execução direta, a sua alternativa. Em execução direta, cada comando seria feito de maneira separada, com circuitos distintos que os executassem, porém, cada instrução deveria ser guardada em um espaço de memória, de modo que essa estratégia seria pesada nela. Com base nessa situação, apresenta-se a micro-programação: para que sejam executados programas normais, a máquina usa micro-programas pré-estabelecidos em uma memória de controle especial, dividindo o problema em vários micro-códigos. Embora fosse lento, no início, o micro-código ainda tinha desempenho aceitável, em razão da memória que o armazenava, diferente da dos códigos comuns.

3. RISC

A micro-programação, embora permitisse implementar instruções complexas em hardware, criou um problema à medida que os microprogramas cresciam, ficando mais difíceis de testar e depurar, resultando em erros que precisavam de correções constantes. Essas dificuldades levaram pesquisadores a questionar se implementar tantas instruções complexas via microcódigo era, de fato, o melhor uso dos recursos disponíveis na época, abrindo caminho para a arquitetura RISC.

Como as implementações CISC eram tão complexas que precisavam de vários chips, buscou-se uma solução de chip único. Isso exigia deixar de fora algumas funcionalidades, então os pesquisadores estudaram quais operações eram mais frequentes na execução real de programas, para otimizar justamente essas.

Esse estudo levou a uma virada de filosofia: em vez de jogar a complexidade do software para o hardware (como fazia o CISC), o RISC devolveu essa complexidade para o software, algo que só era possível porque, àquela altura, a memória havia ficado mais barata e os compiladores, mais eficientes. Na prática, isso significou cortar o suporte a microcódigo e eliminar as instruções complexas que quase ninguém usava, já que um pequeno grupo de instruções simples já dava conta da maior parte do trabalho. Daí vem o nome *Reduced Instruction Set Computer*.

Principais decisões do projeto RISC:

- Uma instrução por ciclo de clock: quase tudo que o microcódigo fazia podia ser feito com instruções assembly simples e rápidas.
- Processadores CISC antigos tinham muito mais instruções e microcódigo bem maior que os primeiros RISC, que não usavam microcódigo algum.

	CISC			RISC	
	IBM370	VAX 11/780	8086	SPARC I	MIPS I
Ano	1973	1973	1978	1981	1983
Nº Instr.	208	303	100	39	55
Microc.	54Kb	400Kb	11Kb	0	0
Instr. (bytes)	2-6	2-57	1-17	4	4

Tabela 1 . diferenças que caracterizaram os primeiros processadores das arquitecturas RISC e CISC

- Pipelining: só é viável quando as instruções têm complexidade uniforme, daí a necessidade de instruções simples e de tamanho fixo.
- Eliminação de modos de endereçamento complexos e aumento de registradores internos: como mais de 80% dos operadores de um programa são variáveis locais, ter muitos registradores evita acessos constantes à memória.
- Separação das instruções LOAD/STORE das demais: apenas essas duas acessam a memória. Todas as outras operações são registrador-registrador. Isso permite ao compilador reorganizar o código para aproveitar os ciclos de espera do LOAD.
- Papel do compilador: no RISC, o compilador se torna muito mais responsável por gerar código otimizado, já que o hardware é mais simples, diferente do CISC, onde essa complexidade fica mais concentrada no hardware.

4. RISC vs CISC

Ao adotar essa filosofia de simplificação, o RISC não apenas reduziu o conjunto de instruções, mas transformou a lógica de onde a complexidade de um processador deveria residir: enquanto o CISC apostava em transferir tarefas para o hardware na forma de instruções ricas e multiciclo, o RISC devolveu essa responsabilidade ao software, confiando em compiladores mais inteligentes para gerar códigos eficientes a partir de instruções simples e rápidas.

Com o passar do tempo as linhas de código cresceram absurdamente e ficaram cada vez mais complexas. A própria Microsoft Windows 95 contém cerca de 10 milhões de linhas de código, a maior parte dele escrito em C. Isso gerou um enorme problema em relação ao início das arquiteturas para agrupar todas linhas em diferentes conjuntos de instruções. A atualização a nível de software ficou muito difícil pois tudo isso ainda seria decodificado para a linguagem de máquina, os 0s e 1s. Para que os computadores rodassem todas essas linhas de código a exigência necessária para o tempo de execução passou a ser estritamente reduzida a 1 ciclo, "batimento" do *clock* do relógio, isso é 40ns a 9 ns, num PENTIUM (fusão entre as duas arquiteturas). Nesse sentido, para aumentar a velocidade de execução do software era preciso alterar a velocidade de um ciclo do clock

do processador.

Para os projetistas de chip na era da microprogramação o objetivo era criar CPUs que executassem instruções em apenas 1 ciclo de clock. Isto é, 1 ciclo por instrução. Enquanto que o objetivo dos projetistas da arquitetura RISC buscavam diminuir a quantidade de instruções por chip e fazer a distribuição necessária de transístores para cada uma. O texto exemplifica como os processadores executam as filosofias iniciais das diferentes arquiteturas em uma instrução de multiplicação 5 x 10.

Na arquitetura CISC ocorrem os seguintes passos:

```
mov ax, 10 ;        Coloca o valor 10 no registrador
ax mov bx, 5 ;      Coloca o valor 5 no registrador
bx mul bx ;         Multiplica ax por bx diretamente no hardware
```

Nesse caso,a instrução `mul bx` é complexa e leva vários ciclos de relógio para terminar dentro do chip, enquanto que na arquitetura RISC:

```
mov ax, 0 ;         Zera o acumulador que guardará o resultado
mov bx, 10 ;        Define o número que será somado (10)
mov cx, 5 ;         Define o contador de repetições (5 vezes)
begin:
add ax, bx ;        Soma 10 ao valor acumulado em ax
loop begin ;        Volta para "begin" e repete a soma 5 vezes.
```

Nesse cenário,o objetivo é reduzir o número de instruções e fazer com que levem apenas 1 ciclo do clock para execução, a fim de evitar consumir muitos transistores e obter circuitos complexos.

5. **Comparação de dois tipos de arquiteturas comerciais:**

O artigo fala ainda que essas instruções realizadas na memória requerem um massivo uso de hardware, pois exige um elevado número de transistores em relação à filosofia inicial. Os processadores RISC se destacam por tentar minimizar essas instruções, além adotar estratégias como cumprimento fixo, melhor performance a nível de operações com ponto flutuante. Isso era bastante atrativo para a comunidade científica da época. Apenas na década de 90 a Intel decidiu adotar a unidade de co-processamento matemático em seus processadores e isso foi um marco pois foi quando de fato os processadores começaram a adotar essas unidades, já implementadas no PENTIUM.

Como a Intel passou a incorporar tecnologias típicas do RISC (*caches* integrados, pipelining e predição de desvios) em seus chips CISC, a presença dessas tecnologias deixou de ser um critério para diferenciar as duas filosofias. A distinção passou a residir em como o *pipeline* lida com a estrutura interna das instruções.

- Pipeline no RISC: apresenta menor variação na estrutura porque possui instruções padronizadas em relação aos registradores e os modos de endereçamento no chip. Essas operações baseadas em registradores internos são simples em relação à decodificação.

- Pipeline no CISC: possui uma estrutura mais complexa pois não tem instruções de tamanho fixo. Elas podem ler ou gravar dados diretamente na memória ram, por isso a decodificação pode demorar mais de um ciclo do relógio.

6. RISC & CISC, lado a lado

O texto narra que a diferença entre as duas arquitetura se tornaram quase imperceptíveis na prática. Poucos processadores mantiveram a linha RISC original. Os fabricantes como AMD, Cyrix,NexGen e a própria Intel (a partir do Pentium II) adotaram a fusão entre as duas. Agora chips convertem internamente instruções CISC complexas em pequenas operações simples de tamanho fixo tipo RISC.

7. O Futuro

O texto fala de uma arquitetura EPIC como a principal evolução RISC/CISC. Foi desenvolvida em conjunto pela Intel e Hewlett-Packard(HP) para os processadores de 64 bits. Ela combina elementos do RISC e CISC, permitindo que o compilador indique claramente quais instruções podem ser executadas em paralelo pelo hardware. No caso dos fabricantes dependentes do mercado x86, o CISC não seria apagado por questões de compatibilidade e para evitar software legado, mas o RISC tradicional poderia, de fato, deixar de ser usado devido ao novo lançamento EPC.

8. Conclusão

Por fim, o artigo “Comparação entre as arquitecturas de processadores RISC e CISC” conclui que, o tamanho ou tipo de instruções não